

LAB 3 & 4

Constructors and Destructors

Default and Parameterized Constructors in C++

Learning Objectives

By the end of this lab, you should be able to:

- Explain what a constructor is and why it is needed.
- Explain what a destructor is and when it is called.
- Write a default constructor that initializes data members automatically.
- Write a parameterized constructor that accepts arguments at object creation.
- Compare object initialization with and without a constructor.

1. The Problem We Are Solving

In Lab 2 you used **setter functions** to assign values to private data members. That works, but every time you create an object you have to remember to call the setter — and if you forget, the variables hold **garbage values**.

What can go wrong without a constructor

You create an object but forget to initialize it → undefined behavior.

Every place that creates the object must remember the same setup steps.

There is no single, guaranteed point where setup happens.



The fix

C++ provides a special function called a constructor that runs automatically every time an object is created. You cannot forget to call it — the language calls it for you.

2. What Is a Constructor?

A constructor is a special member function of a class that has all of these properties:

- **Same name as the class.** If the class is `Student`, the constructor is also named `Student`.
- **Called automatically.** Runs the moment an object is created — you never call it by hand.
- **Used to initialize data members.** Its main job is to give the object a valid starting state.
- **No return type.** Not even `void`. Just write the name and the parameter list.
- **Can be overloaded.** You can have several constructors that take different parameters.

What Is a Destructor?

A destructor is the mirror image of a constructor. It is called automatically when an object is **destroyed** — usually when the object goes out of scope (for example, when `main()` finishes). Its job is to clean up: release memory, close files, free any resources the object was holding.

A destructor:

- Has the same name as the class, prefixed with a tilde (~). Example: `~Student()`.
- Has no return type.
- Has no parameters — and cannot be overloaded.
- Is called automatically when the object's lifetime ends.

Memory aid

Constructor → builds the object up.

Destructor → tears the object down.

Both run automatically. You don't call either one yourself.

3. Code 1 — Default Constructor and Destructor

A **default constructor** takes no parameters. Use it when you want every object to start with the same default values.

The Code

```
#include <iostream>
using namespace std;

class myclass {
    int a;
public:
    myclass();
    void show();
    ~myclass();
};

myclass::myclass() {
    cout << "In constructor!" << endl;
    a = 10;
}

void myclass::show() {
    cout << "Value of a is " << a << endl;
}

myclass::~~myclass() {
    cout << "Destructor called!" << endl;
}
```

```

}

int main() {
    myclass c1, c2;
    c1.show();
}

```

Code Breakdown

Code	What it does
<code>class myclass {</code>	← Define a class named myclass.
<code> int a;</code>	← Private data member by default (members of a class are private unless stated otherwise).
<code>public:</code>	← Everything below is accessible from outside.
<code> myclass();</code>	← DECLARES the default constructor — no parameters, no return type.
<code> void show();</code>	← A regular member function — declared here, defined below.
<code> ~myclass();</code>	← DECLARES the destructor — note the tilde (~).
<code>myclass::myclass() {</code>	← DEFINITION of the constructor (the :: tells the compiler 'this belongs to myclass').
<code> cout << "In constru..";</code>	← We print a message so we can SEE when the constructor runs.
<code> a = 10;</code>	← Initialize the data member to a known starting value.
<code>myclass::~~myclass() {</code>	← Definition of the destructor — runs automatically when the object dies.
<code> cout << "Destructor..";</code>	← Prints so we can SEE when the destructor runs.
<code>myclass c1, c2;</code>	← Create TWO objects. Constructor runs ONCE for each — c1 first, then c2.
<code>c1.show();</code>	← Call show() on c1. (We never call show() on c2.)
<code>}</code>	← End of main. Both objects go out of scope here, so destructors run.

Expected Output

► Output

```

In constructor!
In constructor!
Value of a is 10
Destructor called!
Destructor called!

```

Read the output carefully

Two "In constructor!" lines appear before any other output — proof that constructors fire automatically when c1 and c2 are created.

Only one "Value of a is 10" line, because show() was called only on c1.

Two "Destructor called!" lines appear at the end — one for each object. They are called in REVERSE order of creation: c2 dies first, then c1. This is called LIFO (Last In, First Out).

Compiler-generated default constructor

If you don't define ANY constructor, the compiler quietly generates a do-nothing default one for you. The moment you define your own constructor, that free one disappears — so define one explicitly when you need predictable behavior.

4. Code 2 — Parameterized Constructor

A **parameterized constructor** takes arguments at the moment of object creation. This lets you build different objects with different starting values, without writing setter functions.

The Code

```

#include <iostream>
using namespace std;

class Student {
private:
    int id;
    string name;
public:
    Student(int i, string n);
    void display();
};

Student::Student(int i, string n) {
    id = i;
    name = n;
}

void Student::display() {

```

```

        cout << "ID: " << id << ", Name: " << name << endl;
    }

    int main() {
        Student s1(101, "Rahim");
        Student s2(102, "Karim");

        s1.display();
        s2.display();

        return 0;
    }

```

Code Breakdown

Code	What it does
private:	← id and name are hidden — encapsulation in action.
int id;	← First data member.
string name;	← Second data member.
Student(int i, string n);	← DECLARES a constructor that takes TWO parameters.
Student::Student(int i, string n) {	← DEFINITION — note the parameters match the declaration.
id = i;	← Copy the parameter into the data member.
name = n;	← Same idea for name.
Student s1(101, "Rahim");	← Constructor runs IMMEDIATELY with id=101, name="Rahim".
Student s2(102, "Karim");	← Same constructor runs again with different values for s2.
s1.display();	← Print s1's data.
s2.display();	← Print s2's data — proves the two objects hold different values.

Expected Output

► Output

```

ID: 101, Name: Rahim
ID: 102, Name: Karim

```

 One constructor, many objects

The same `Student(int, string)` constructor was used to create both `s1` and `s2` — but each object stores its own private copy of `id` and `name`. That is the whole point of objects: shared behavior, separate state.

 Don't forget `#include <string>`

Whenever you use the `string` type, add `#include <string>` at the top. Many compilers will let it slide because `<iostream>` indirectly pulls it in, but relying on that is fragile — make it explicit.

5. Code 3 — Why Constructors Make Code Cleaner

This code uses the same `Student` class to create just one object. Notice how short `main()` stays — initialization happens in a single line, right where the object is created.

The Code

```
#include <iostream>
using namespace std;

class Student {
private:
    int id;
    string name;
public:
    // Constructor declaration
    Student(int i, string n);
    void display();
};

Student::Student(int i, string n) {
    id = i;
    name = n;
}

void Student::display() {
    cout << "ID: " << id << endl;
    cout << "Name: " << name << endl;
}

int main() {
    Student s1(101, "Rahim");
    s1.display();
    return 0;
}
```

Code Breakdown

Code	What it does
<code>Student(int i, string n);</code>	← Constructor declared inside the class — same idea as Code 2.
<code>Student::Student(...) {</code>	← Constructor body — runs automatically at object creation.
<code>void display() {</code>	← This version prints id and name on TWO separate lines (different format from Code 2).
<code>Student s1(101, "Rahim");</code>	← Single line: declare, allocate, initialize, all at once.
<code>s1.display();</code>	← Print the data — no extra setup steps in main.

Expected Output

► Output
<pre>ID: 101 Name: Rahim</pre>

With Constructor vs Without Constructor

If you removed the constructor and used setter functions instead (as in Lab 2), you would notice these differences:

Feature	Without Constructor	With Constructor
Initialization	Manual (must call a separate setter)	Automatic (done in constructor)
Risk of error	High (you may forget to initialize)	Low (initialization always happens)
Code simplicity	Extra setter functions needed	Cleaner code
OOP approach	Less object-oriented	Follows OOP principles
Readability	Lower	Higher
Maintenance	Harder	Easier

 **Bottom line**

A constructor makes object creation SAFE BY DEFAULT. The compiler enforces it — you literally cannot create an object without going through the constructor. That guarantee is what makes constructors more reliable than setter functions.

6. Summary

Four key ideas to take away from this lab:

1. **Constructors run automatically.** You never call them by hand — creating an object IS the call.
2. **Default vs parameterized.** Default takes no arguments; parameterized takes one or more arguments to customize the new object.
3. **Destructors clean up.** They run when the object dies (end of scope), and run in REVERSE order of construction.
4. **Initialization beats setters.** Constructors guarantee setup. Setters rely on the programmer remembering to call them.

7. Practice Task

Task 1 — Circle

Create a class Circle with the following:

Private data member:

- radius (float)

Public member functions:

- A parameterized constructor that takes the radius as an argument and stores it.
- area() — returns $\pi \times r^2$.
- circumference() — returns $2 \times \pi \times r$.
- A way to display the radius, area, and circumference of the circle.

In main():

- Create at least 2 Circle objects with different radii.
- Display the radius, area, and circumference for each.

Hints

Use 3.14159 (or define `const float PI = 3.14159;`) for π .

Use float — not int — so the area and circumference can hold decimal results.

If you want extra practice: add a destructor that prints "Circle with radius X destroyed" so you can see destruction happen at the end of main().

Common mistakes to avoid

Putting a return type on the constructor (it has none — not even void).
Forgetting that the constructor's name must match the class name EXACTLY (case-sensitive).
Trying to call the constructor manually — never write `s1.Circle(5)`; the compiler calls it for you when you write `Circle s1(5);`.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST